TinyAstral

Transferring Self-Evolving Harnesses to Tiny Generalist Manipulation Agents

Paper Review • Kehong Gong et al. • Huawei Technologies / Noah Research Institute • arXiv 2610.41216

One-line summary: Self-evolve a large teacher's harness under statistical gating, then distill its reasoning and recovery behaviors into a 4B model, which nears teacher performance with ~1/5 the accelerator time.

33.4% → 63.6%

Teacher RoboTwin SR, evolved harness

62.8%

TinyAstral-4B student (same harness)

4.9×

Teacher:student deploy accel-s

Robot agent = planner + harness; both drive performance

Planner: decides "what to do"

Which object and part, reference objects and spatial relations

Which arm, in what order, when to finish (call done)

Each turn: one reasoning step + exactly one tool call; no joint angles/coordinates

Harness: decides "how to do it"

Perception: SAM 3 segmentation + depth point clouds

Grasp/place geometry, collision-free motion planning, closed-loop correction

Tools return measured outcomes (e.g., OFF_TARGET + offset), task memory, recovery logic

Two existing approaches, each with shortcomings

Harness evolution

Freeze the model, modify the harness (PhysEvo, HarnessPAI, RoboRSI…)

But gains are tied to large models with costly inference

Agent distillation

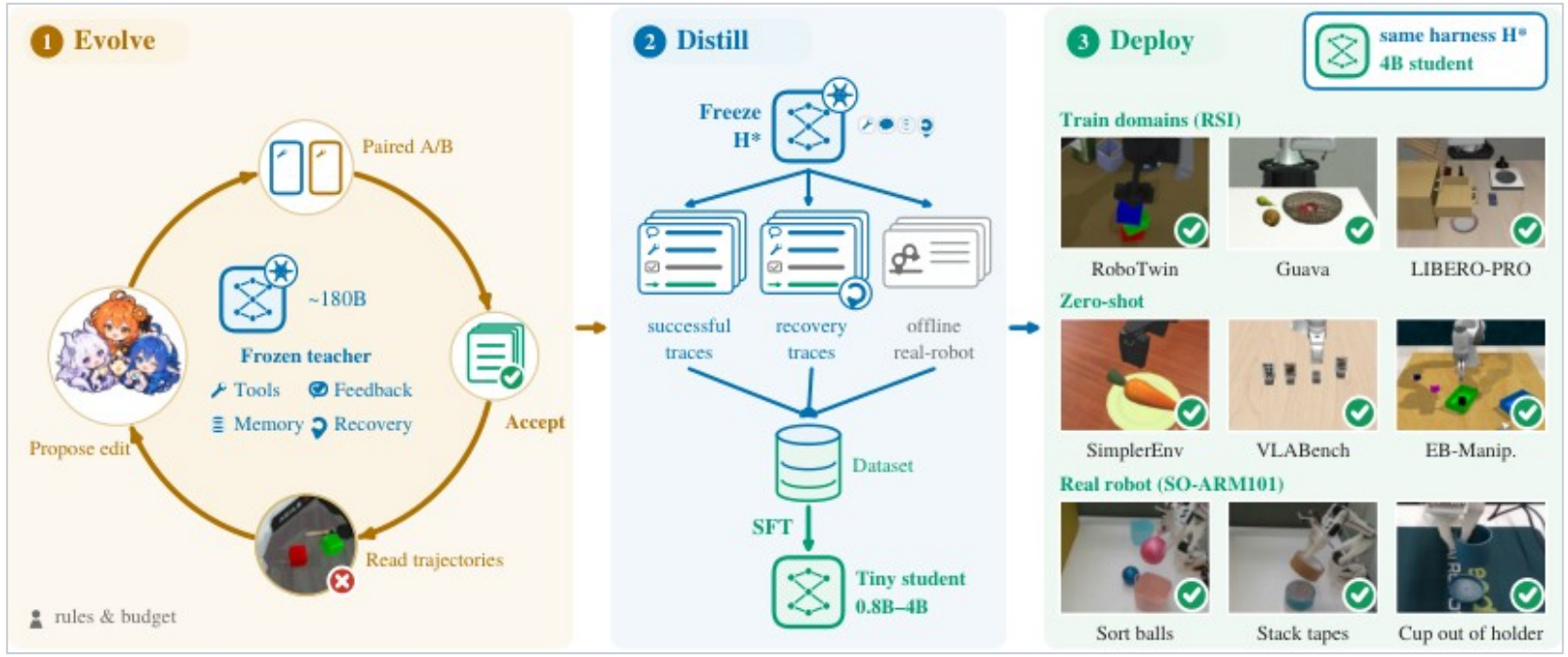
Train small models on strong-model trajectories (Guava, World Action Agent…)

But the teacher harness is usually static

Research question

If a large teacher's harness is evolved first, do the gains transfer to a small model via distillation, enabling low-cost deployment?

Two stages: evolve harness → freeze → distill a small student



① Evolve

Teacher weights frozen; a frontier LLM "master" reads trajectories and proposes harness edits, accepted only if a paired A/B test passes

② Distill

Freeze final H*; collect successful trajectories + recovery branches + offline real-robot data; SFT 0.8B–4B students

③ Deploy

Student replaces teacher in the same H*; evaluated on 3 training domains + 3 zero-shot benchmarks + real SO-ARM101

Stage 1: harness edits must pass preregistered paired tests

1. Read traces

Run H_k on dev seeds; sort failures into 10 classes (misplacement, false completion, missing primitive, memory error, wrong instance…)

2. Propose edits

New tools/primitives, more honest returns, perception fixes, failure handling, local task-memory edits

3. Paired test

Candidate H′ and its parent run interleaved in parallel on ~1,150 preregistered (task, seed) pairs

Accept / reject rules

Accept: task-clustered bootstrap one-sided 97.5% lower bound of paired gain on triggered set $J > 0$, and target-cluster net wins > 0

Reject: untriggered set U regresses $\geqslant$ 2 pp (two-sided 95% upper bound < 0)

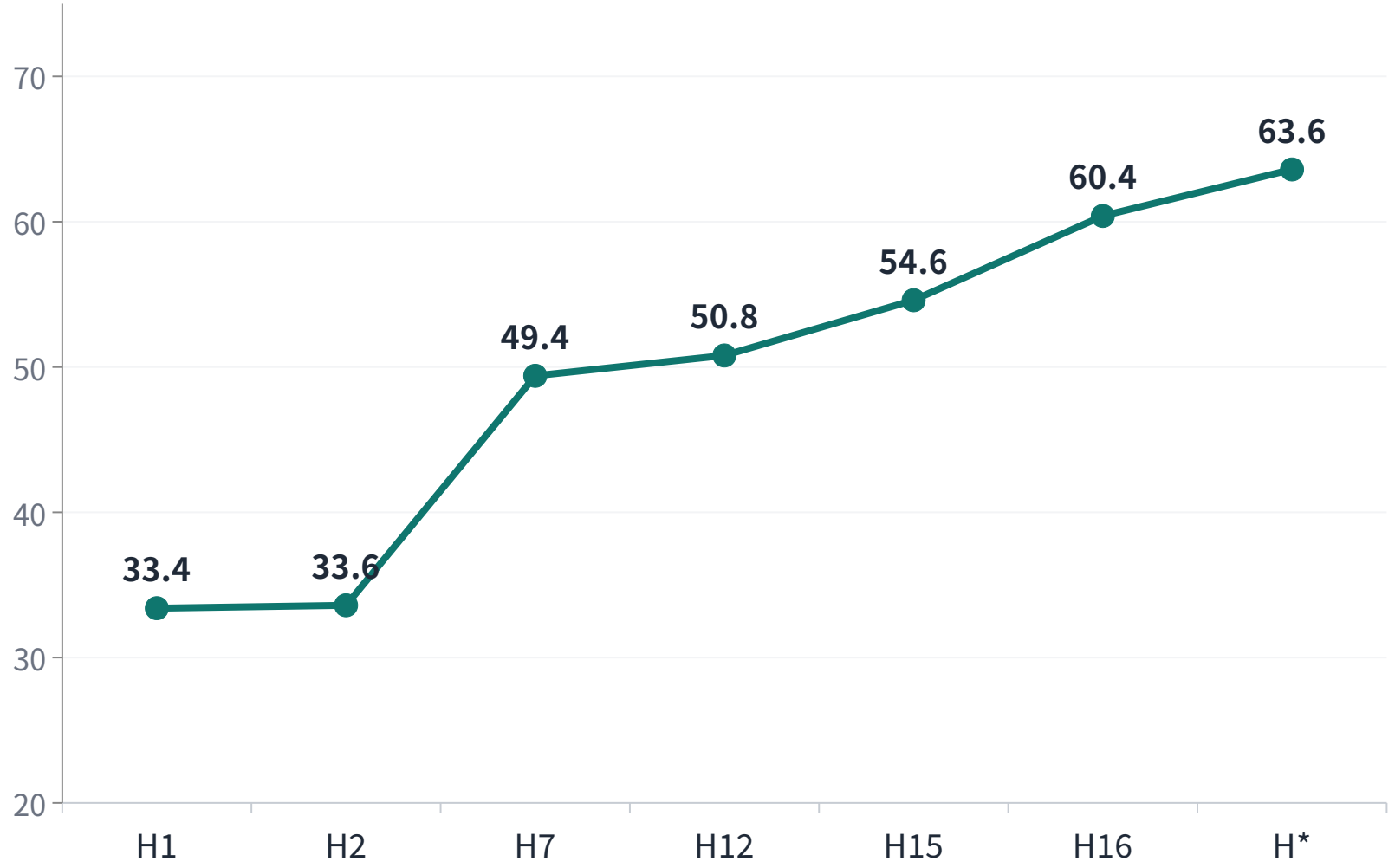
Pre-screen: direction filter (discard if target-cluster gain 95% upper bound < +2 pp); interim futility stopping

Anti-gaming: no branching on task names; system prompt and existing tool descriptions byte-identical; new components need explicit trigger + log marker + code audit

Capability ledger (all runs)

296	Candidates
121	Adopted
142	Rejected
101	In final H*

Harness edits alone nearly double teacher RoboTwin success



+30.2 pp

H1 → H* : 33.4% → 63.6%

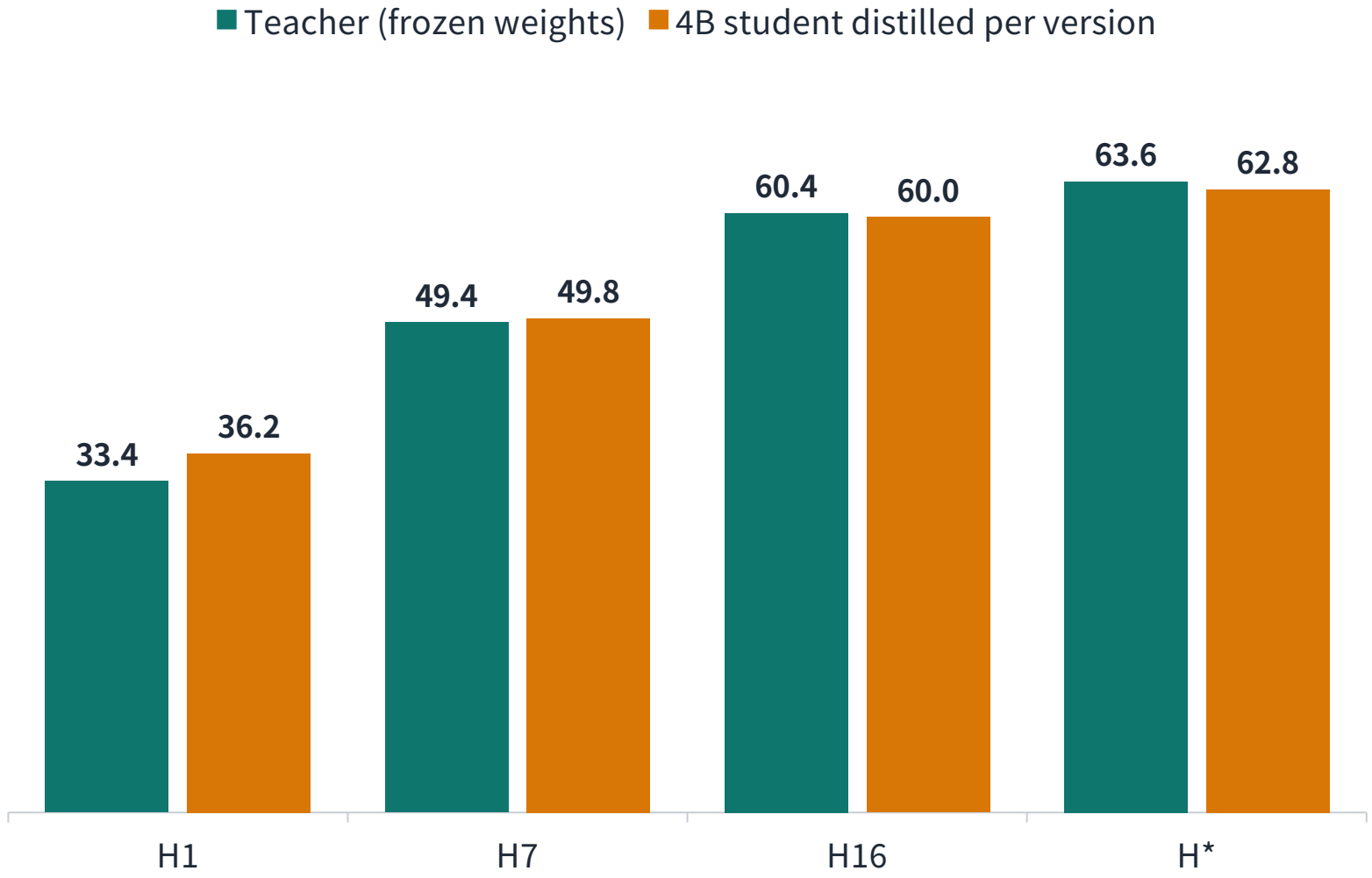
Notes

H_k is the k-th accepted mainline snapshot

Final integration stage: 3 of 7 candidates accepted

Task memory is RoboTwin-only: one end-state note per task, no actions, read-only at eval

Evolution gains transfer: each student tracks its teacher



Cross-planner transfer

DeepSeek-V4.1-Flash （552B）

Not used in evolution; H1→H* +30.6 pp

Untrained Qwen3.5-4B

Also benefits: 14.6% → 41.2%

No model-specific harness changes

Value of distillation

Both under H*: 62.8% vs 41.2%

Distilled is 21.6 pp above untrained 4B

Stage 2: reasoning + recovery + strict success + real-robot aid

Keep teacher reasoning

Target = <think>…</think> + next tool call; loss only on the final assistant turn

Recovery branches

Model-free replay of a successful trajectory to step k, inject perturbations (slip, target moved 6–10 cm, empty grasp, incomplete push), teacher continues online; keep only branches with effective perturbation, consistent replay and strict success

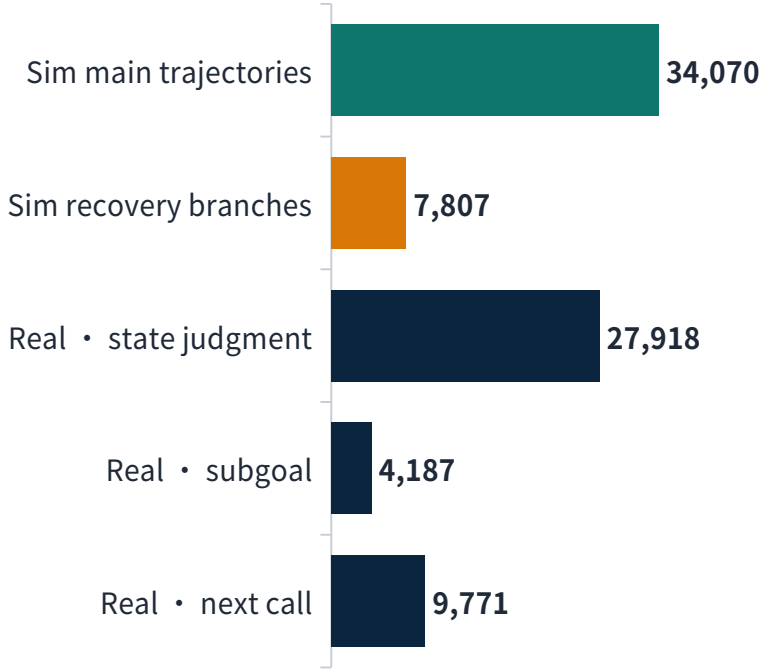
Strict successes only

Temperature 0; failed seeds retried up to twice at 0.7; must end with done in a successful final state; "succeeded, then broke it" discarded; success labels kept out of training messages

Offline real-robot aid

AgiBot World, Fractal, Bridge V2, DROID, UR5 converted to harness format; reasoning via STaR-style rationalization with quality checks

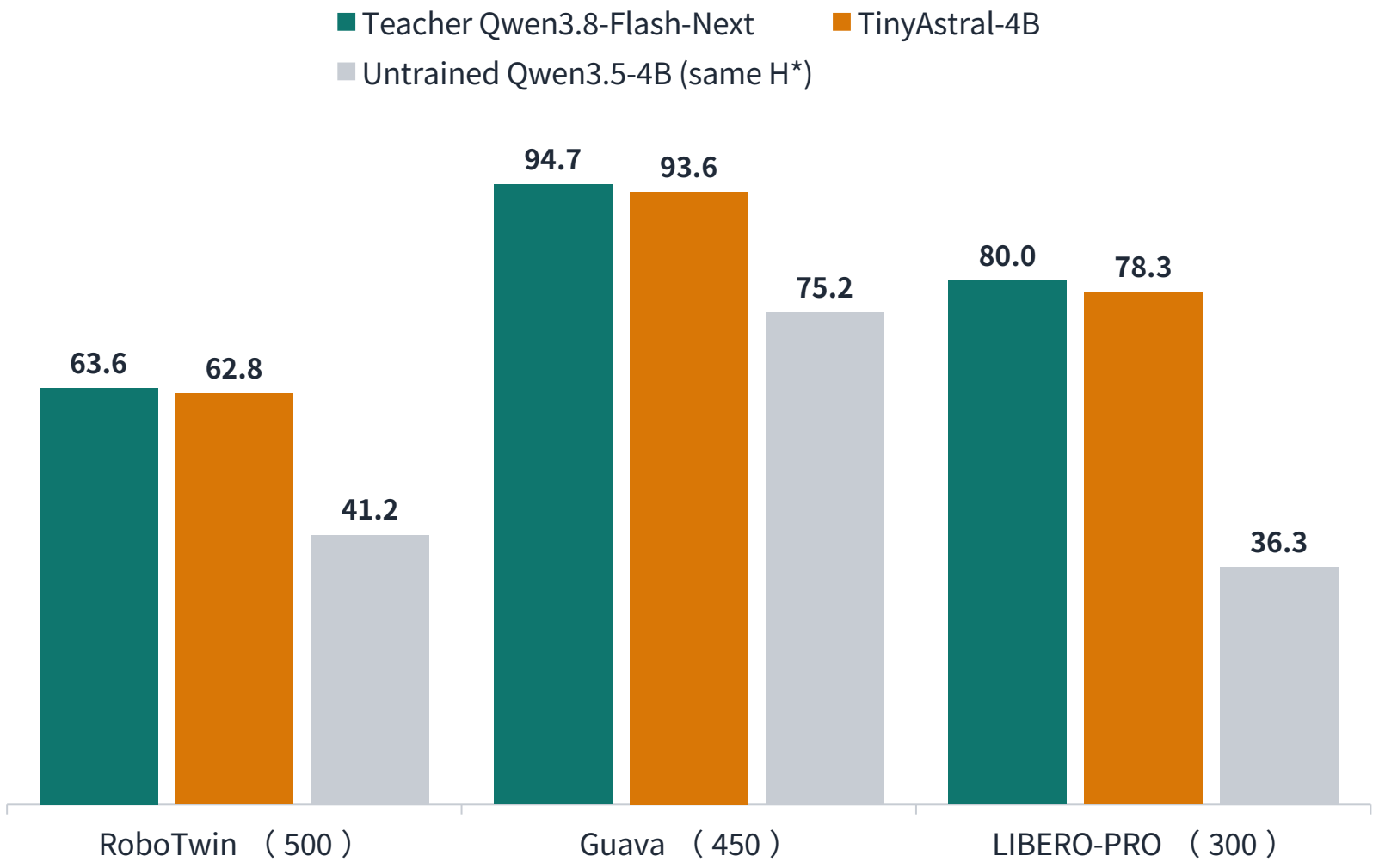
Data: 83,753 decision samples



Sim 41,877: RoboTwin / Guava / LIBERO main trajectories 13,857 / 9,054 / 11,159; recovery branches 3,471 / 2,048 / 2,288

Full fine-tuning of the Qwen3.5-4B language model, 2 epochs, lr 1e-5, vision encoder frozen

Training domains: 4B student within 1.7 pp of teacher



−0.8 / −1.1 / −1.7

Student gap vs. teacher (pp)

+18.4 / +42.0

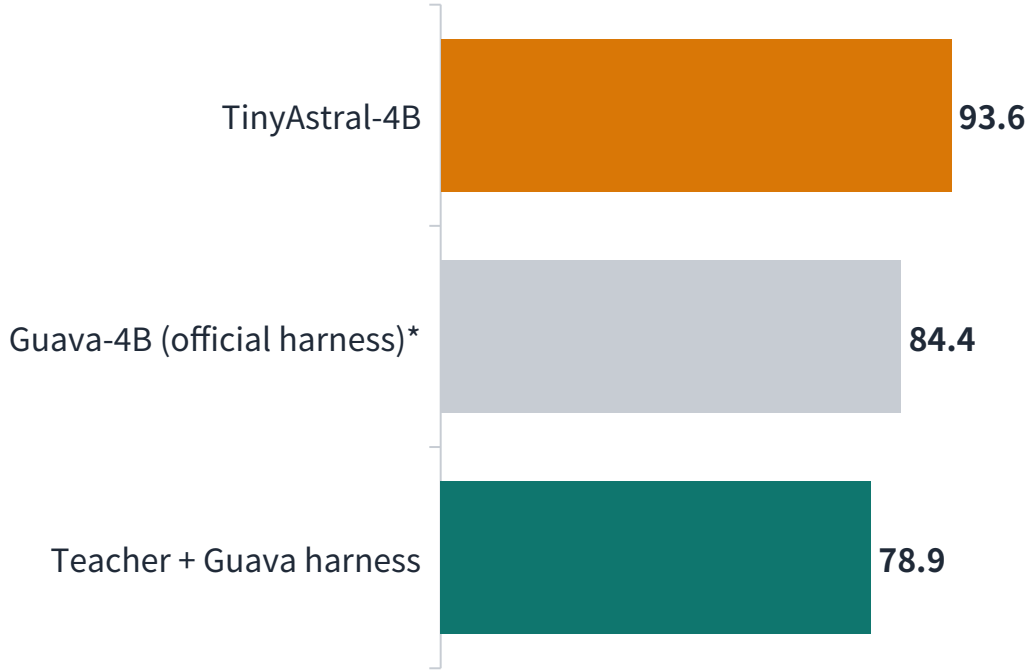
vs. untrained 4B: Guava / LIBERO-PRO (pp)

Even counting all undecided outcomes as successes, untrained 4B reaches at most 57.3% on LIBERO-PRO

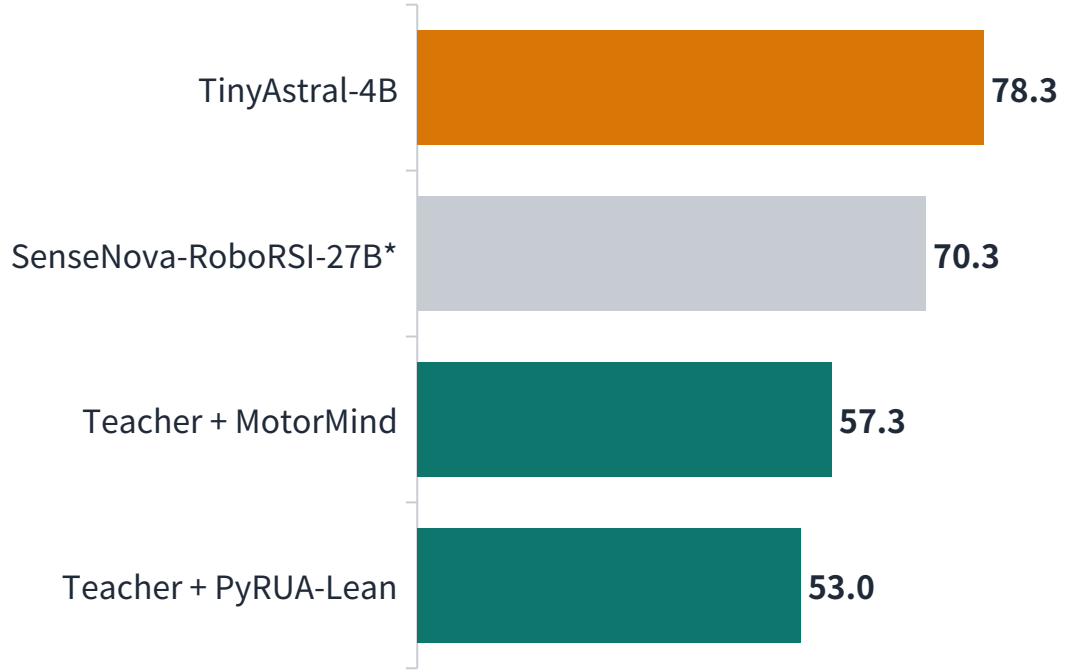
LIBERO-PRO training data comes only from LIBERO-90

Same episodes: beats open models and teacher in rival harnesses

Guava (15 tasks × 30 seeds)



LIBERO-PRO （300 episode）



RoboTwin: PyRUA-Lean-4B only 12.8% (64/500)

Its training/self-assessment relies on per-step env success signals; absent in this protocol, it often declares success prematurely

Note: the same teacher does far worse in rival harnesses

→ Gains stem from the evolved harness + distillation, not the base model alone (competitor protocols in Appendix B)

Zero-shot: 4B student within 2.4 pp of the 552B planner

SimplerEnv Google VM

66.6%

Published π0-FAST 61.9 (Fractal fine-tuned)

SimplerEnv WidowX

88.5%

Published π0-FAST 32.1 (Bridge fine-tuned)

VLABench (500 episode)

41.2%

Published π0 29.4 / π0-FAST 43.1 (VLABench fine-tuned)

EB-Manipulation (228 episode)

27.6%

GPT-4o 28.9 (GT coordinates & boxes + ICL examples)

⩽ 5.0 pp

Gap to teacher

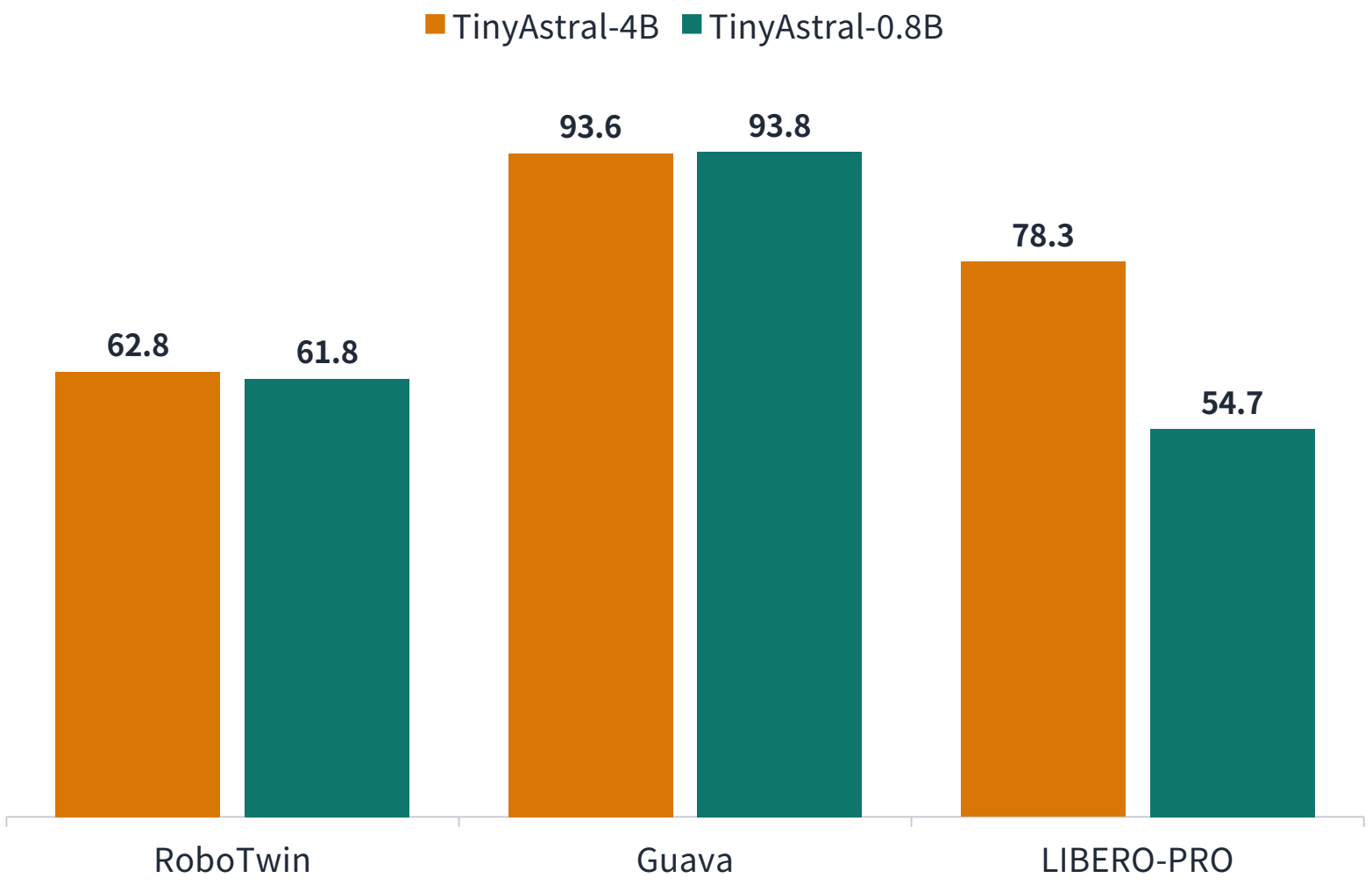
⩽ 2.4 pp

Gap to 552B DeepSeek-V4.1-Flash

+10.0 / +13.6

vs. untrained 4B: Google VM / VLABench (pp)

0.8B barely drops on most benchmarks, except LIBERO-PRO



−23.6 pp

0.8B drop on LIBERO-PRO (78.3 → 54.7)

⩽ 3.0 pp

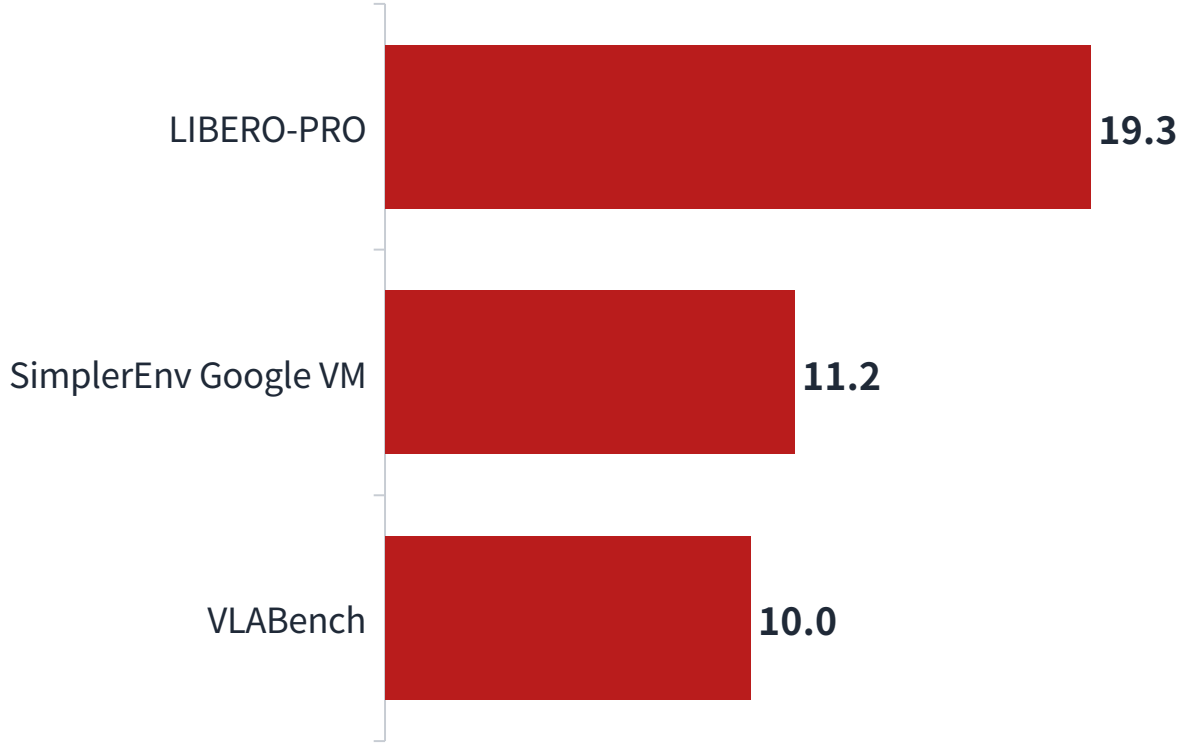
0.8B / 2B gap to 4B across 4 zero-shot settings

0.8B, 2B and 4B use the same data and recipe

Scale matters most on LIBERO-PRO (language/spatial perturbations)

Ablation: teacher reasoning is the key distillation factor

Drop from removing reasoning text, all else unchanged (pp)



LIBERO-PRO episodes exhausting the call limit: **49 / 300** (no reasoning) vs **7 / 300** (full recipe)

Real-robot data: acts on real images

Next-call accuracy on held-out real-robot episodes

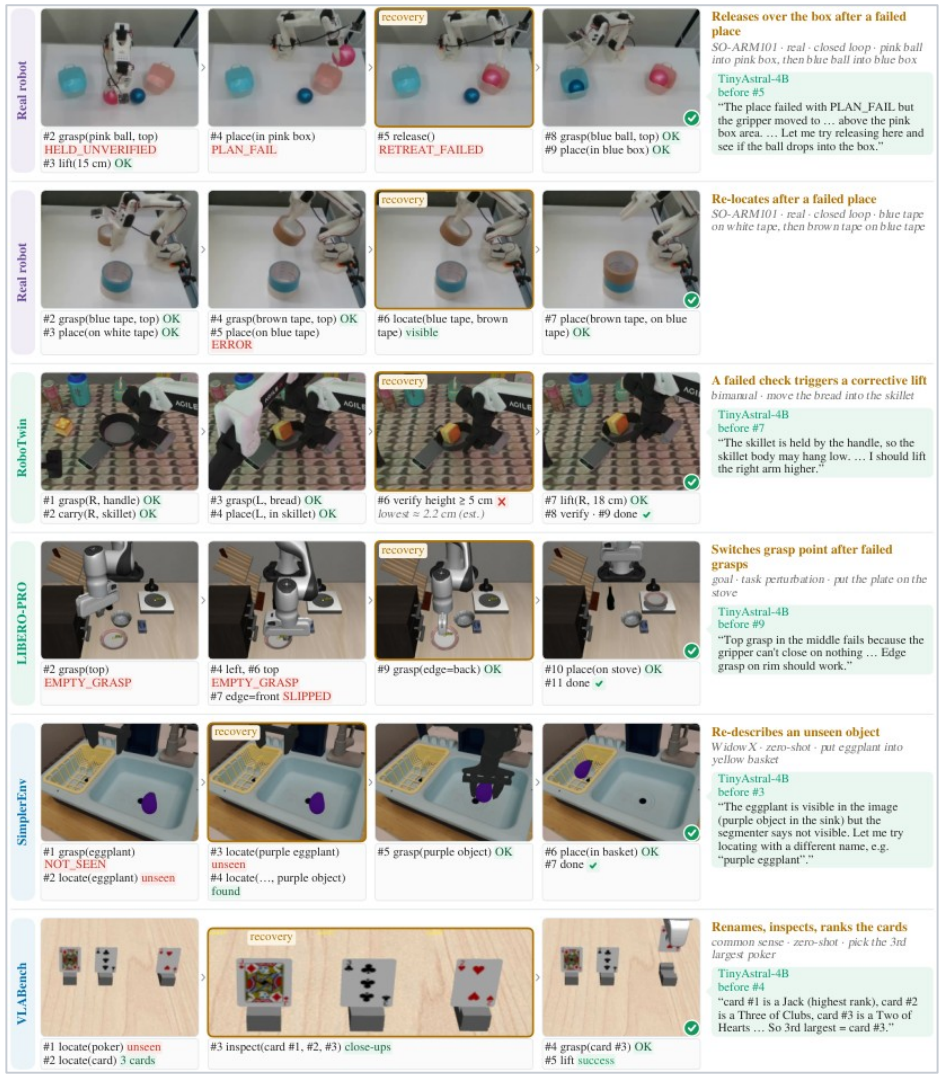
0.235 → 0.923

But on sim success…

No statistically detectable change across 7 sim settings

→ Real-robot data aids offline decisions on real observations, not sim scores

Students self-recover: rephrase, regrasp, use measured feedback



Real SO-ARM101

place returns PLAN_FAIL → release above the box; re-locate after failed placement

Training domains

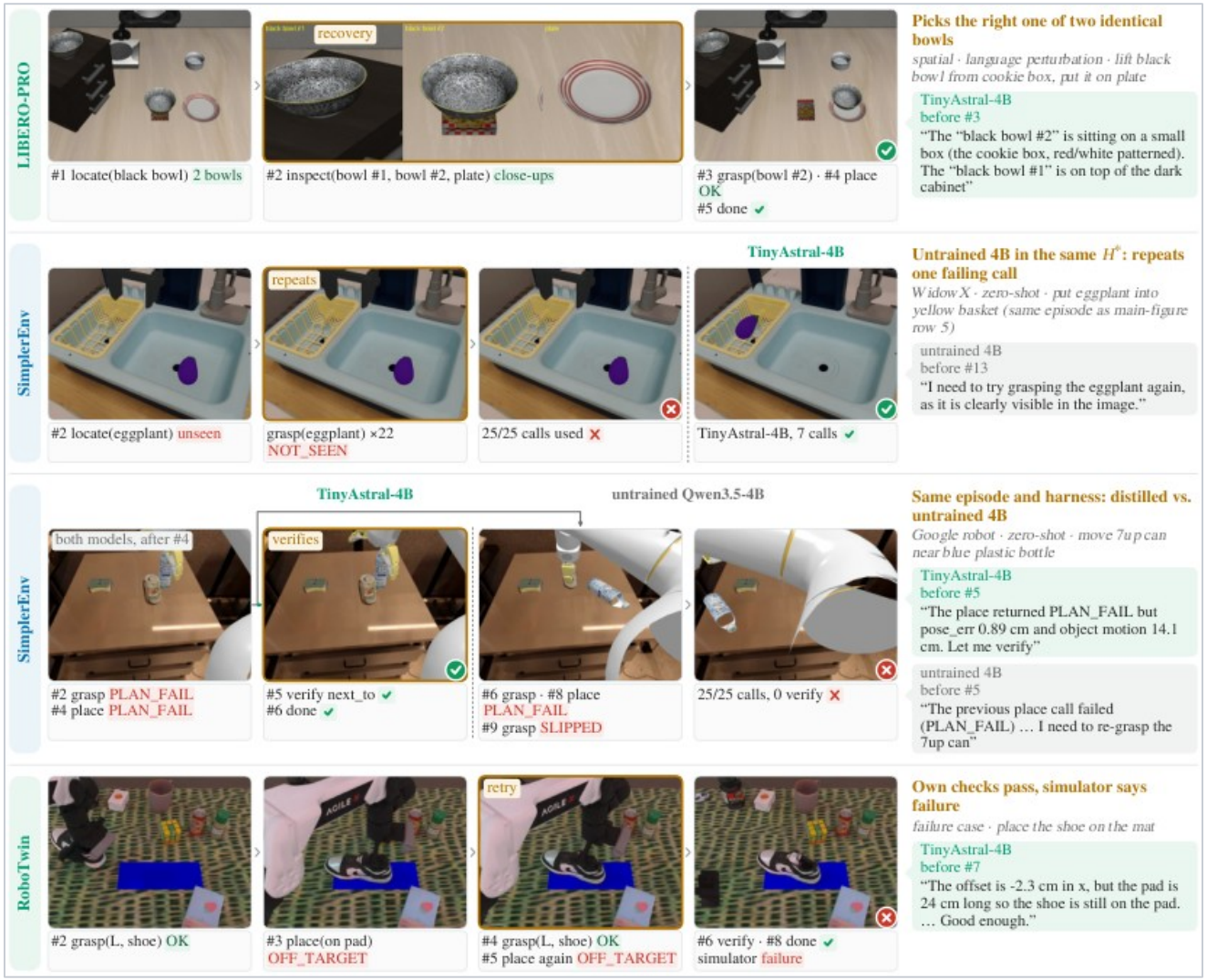
verify finds insufficient height → corrective lift; top grasp EMPTY_GRASP → grasp the edge

Zero-shot

eggplant NOT_SEEN → rephrased as "purple object"; VLABench: close-up card check before sorting

Top to bottom: real robot ×2 / training domain ×2 / zero-shot ×2

Untrained 4B mostly fails by burning budget on futile calls



171 / 271

Of zero-shot episodes the student solves but untrained 4B fails, 63% stem from exhausted call budgets

Typical contrast (row 2)

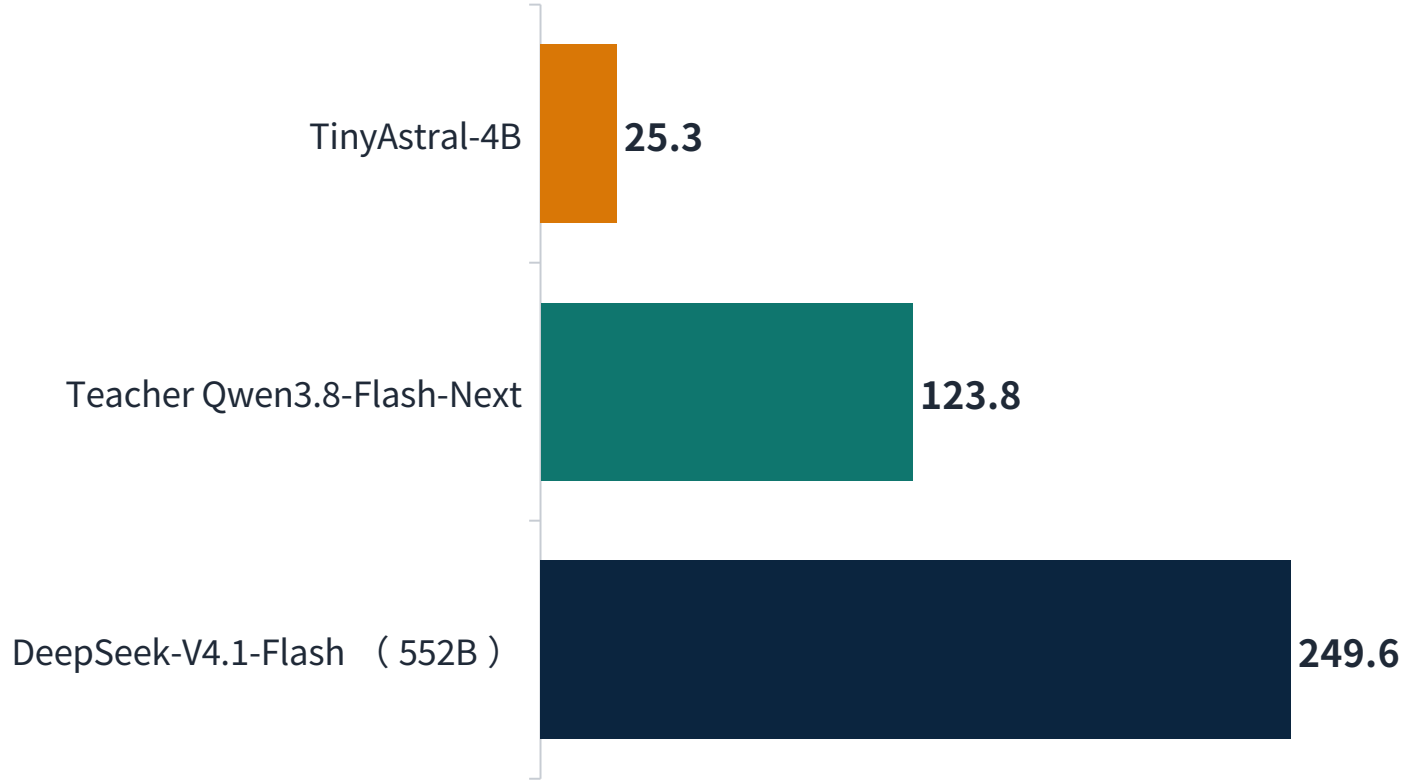
Untrained 4B repeats grasp(eggplant) 22 times (NOT_SEEN) until the budget runs out; the student rephrases the target and succeeds in 7 calls

Student failure (row 4)

Shoe OFF_TARGET by 2.3 cm; student self-check passes and declares success, but the simulator judges failure: self-check tolerance mismatches the benchmark criterion

Cost: ~1/5 the teacher's and 1/10 the 552B's accelerator time

Accelerator-seconds / episode (accelerators × median duration)



Median episode time (incl. sim & perception): student 25.3 s, teacher 31.0 s, DeepSeek 31.2 s

4.9×
vs teacher

9.9×
vs DeepSeek

Deployment (same accelerators)

Student: 1 device, bf16, no quantization or speculative decoding

Teacher: 4 devices, FP8 + tensor parallelism + speculative decoding

DeepSeek-V4.1-Flash: 8 devices + speculative decoding

Caveat: only 20 episodes; wall-clock gaps are small, savings lie mainly in accelerator usage

Verdict: solid, but scoped to high-level tool-call harnesses

Strengths

Multi-snapshot distillation: students of all four versions track their teachers, directly showing that "evolution gains transfer"

Rigorous RSI statistics: preregistered tests, paired interleaving, trigger audits, three-way seed split, full ledger

Relatively fair evaluation: no success signal, strict success, fixed denominators, same-episode pairing, same teacher in rival harnesses

Practical: one 4B model covers 6 benchmarks + real robot, deployed on a single device in bf16

Limitations & questions

RSI and task memory used only on RoboTwin; sources of gains on other benchmarks not decomposed

Perception, geometry and motion planning all live in the harness; "4B≈552B" holds only under such a high-level interface

Only strict successes distilled, so the student is capped by the teacher; surpassing the teacher not shown

master relies on a frontier LLM; total RSI cost unreported; cost measured on only 20 episodes

Self-check tolerance may misjudge success; real-robot demos cover few tasks, lacking large-scale statistics

Takeaway

1. **Harness is a first-class citizen:** harness-only edits with frozen weights lift teacher RoboTwin success 33.4% → 63.6%, and also help other planners

2. **Evolution gains are distillable:** 4B students track their teachers across all four harness versions; final 62.8% vs 63.6%

3. **Small model suffices and is cheap:** zero-shot gap to 552B ⩽ 2.4 pp; ~1/5 the teacher's deployment accelerator time

4. **Reasoning is key to distillation:** removing reasoning text costs up to 19.3 pp

arXiv 2610.41216 • TinyAstral （Huawei, 2026-10-30）